

ソフトウェア演習 **I**

第2回

塚田 武志

千葉大学

C言語入門

コンパイル・実行

(サンプルプログラム hello.c をダウンロード)

コンパイル

- プログラムを計算機が解釈できる形式に翻訳すること

```
$ cc hello.c
```

a.out というファイルが作られる

実行

```
$ ./a.out
```

c プログラムのコンパイル

次の段階に分けられる

- **プリプロセス** `$ cc -E hello.c` → 標準出力
 - 定数やライブラリ宣言のための前処理
- **狭義の コンパイル** `$ cc -S hello.c` → `hello.s`
 - c 言語のプログラムをアセンブリ言語のプログラムに翻訳
- **アセンブル** `$ cc -c hello.s` → `hello.o`
 - 再配置可能な機械語ファイル（オブジェクトコード）に変換
- **リンク** `$ cc hello.o` → `a.out`
 - オブジェクトファイルやライブラリをまとめ、実行可能ファイルを作成

C プログラムの例

```
/* hello.c -- say `Hello.' NTIMES times. */
```

コメント
プリプロセッサが
取り除く

```
#include <stdio.h>
```

printf 関数を利用するためのおまじない

```
#define NTIMES 10
```

マクロ（プリプロセッサが NTIMES を 10 に置換）

```
int main()
```

関数 main の定義. プログラムはここから実行される

```
{
```

```
    int i;
```

```
    i = 0;
```

```
    while (i < NTIMES) {  
        printf("Hello.¥n");
```

```
        i = i + 1;
```

```
    }
```

```
    return 0;
```

```
}
```

C言語が Python と違うところ

- 改行・インデントに意味がない
- 変数は使う前に宣言が必要
- 変数・関数は宣言した型に従って使う
- 配列の長さは宣言時に決める

などなど

C言語が Python と違うところ

- 改行・インデントに意味がない
- 変数は使う前に宣言が必要
- 変数・関数は宣言した型に従って使う
- 配列の長さは宣言時に決める

色々違うけど、
なんとなく意味は分かるはず！

などなど

C言語が Python と違うところ

- 改行・インデントに意味がない
- 変数は使う前に宣言が必要
- 変数・関数は宣言した型に従って使う
- 配列の長さは宣言時に決める

などなど

改行・インデントに意味がない

空白を入れられる場所なら どこでも好きなだけ改行・インデントができる

- 代わりに文の終わりを「;」で明示

C言語

次の2つは区別されない

```
i = i + 1;  
s = s + i;
```

```
i = i + 1;  
s = s + i;
```

Python

上はよいが, 下はエラー

```
i = i + 1  
s = s + i
```

```
i = i + 1  
    s = s + i
```

改行・インデントに意味がない

空白を入れられる場所なら どこでも好きなだけ改行・インデントができる

- 代わりに文の終わりを「;」で明示

C言語

次の2つは区別されない

```
i = i + 1;  
s = s + i;
```

```
i = i  
+ 1;  
s = s + i;
```

Python

2つとも合法だが、意味が違う

```
i = i + 1  
s = s + i
```

```
i = i  
+ 1  
s = s + i
```

改行・インデントに意味がない

空白を入れられる場所なら どこでも好きなだけ改行・インデントができる

- 代わりに文の終わりを「;」で明示

C言語

n の偶奇に関わらず

n=n+1 を実行

```
if(n % 2 == 0)
    n = n / 2;
else
    n = 3 * n;
    n = n + 1;

printf("%d¥n", n);
```

Python

n=n+1 が実行されるのは

n が奇数のときだけ

```
if n mod 2 == 0:
    n = n // 2
else:
    n = 3 * n
    n = n + 1

print(n)
```

改行・インデントに意味がない

空白を入れられる場所なら どこでも好きなだけ改行・インデントができる

- 代わりに文の終わりを「;」で明示

C言語

n の偶奇に関わらず

n=n+1 を実行

```
if(n % 2 == 0)
    n = n / 2;
else
    n = 3 * n;
n = n + 1;

printf("%d¥n", n);
```

Python

n=n+1 が実行されるのは

n が奇数のときだけ

```
if n mod 2 == 0:
    n = n // 2
else:
    n = 3 * n
    n = n + 1

print(n)
```

改行・インデントに意味がない

空白を入れられる場所なら どこでも好きなだけ改行・インデントができる

- 代わりに文の終わりを「;」で明示

C言語

右の Python プログラムと同じ

```
if(n % 2 == 0){  
    n = n / 2;  
} else {  
    n = 3 * n;  
    n = n + 1;  
}  
printf("%d¥n", n);
```

Python

n=n+1 が実行されるのは
n が奇数のときだけ

```
if n mod 2 == 0:  
    n = n // 2  
else:  
    n = 3 * n  
    n = n + 1  
  
print(n)
```

C言語が Python と違うところ

- 改行・インデントに意味がない
- **変数は使う前に宣言が必要**
- 変数・関数は宣言した型に従って使う
- 配列の長さは宣言時に決める

などなど

変数の宣言

C 言語では変数は使う前に**宣言**が必要

C言語

```
int sum(int n)
{
    int s;
    int i;
    i = 0;
    s = 0;
    while(i <= n){
        s = s + i;
        i = i + 1;
    }
    return s;
}
```

Python

```
def sum(n):

    i = 0
    s = 0
    while(i <= n):
        s = s + i
        i = i + 1

    return s
```

変数の宣言

C 言語では変数は使う前に宣言が必要

C言語

```
int sum(int n)
{
    int i;
    i = 0;
    s = 0;
    while(i <= n){
        s = s + i;
        i = i + 1;
    }
    return s;
}
```

コンパイルエラー

Python

```
def sum(n):

    i = 0
    s = 0
    while(i <= n):
        s = s + i
        i = i + 1

    return s
```


C言語が Python と違うところ

- 改行・インデントに意味がない
- 変数は使う前に宣言が必要
- **変数・関数は宣言した型に従って使う**
- 配列の長さは宣言時に決める

などなど

変数・関数の型

変数・関数は宣言時に**型**を明らかにする

```
int sum(int n)
{
    int s;
    int i;
    ...
}
```

sum は int 型（整数型）の引数を 1 つ取り
整数型の値を返す

変数 s と i に格納されるのは
int 型（整数型）の値

関数定義および関数・変数の使用は宣言した型に従う

```
...
r = sum(21, 0);
...
```

コンパイルエラー

どんな型があるか

- `int`

整数型

32bit であることが多い

- `char`

文字格納用の小さい 整数型

8bit であることが多い

- `float`

浮動小数点数

次の型については後述

- 各型を要素型とする 配列
- 各型への ポインタ

注意

型が合わない操作をすると（必ずしもエラーを起こさず）型変換が試みられる

- 意図しない挙動をすることがあるので気をつける

```
#include <stdio.h>
int main()
{
    int i;
    i = 0.2;
    if(i > 0.1){
        printf("i > 0.1\n");
    } else {
        printf("i <= 0.1\n");
    }
    return 0;
}
```

0.2 を整数で近似して i に代入

「i <= 0.1」が表示される

C言語が Python と違うところ

- 改行・インデントに意味がない
- 変数は使う前に宣言が必要
- 変数・関数は宣言した型に従って使う
- 配列の長さは宣言時に決める

などなど

配列の長さは宣言時に決める

```
int fib(int n)
{
    int a[20];
    ...
    a[0] = 0;
    ...
    a[i] = a[i-1] + a[i-2];
    ...
}
```

a は int 型の要素を持つ長さ 20 の配列
(a[0], a[1], ..., a[19] が使える)

Python と異なり、**作った後で伸ばしたり・縮めたりはできない**

- 必要なら「データを一系列に並べた伸び縮みさせられる構造」を自分で作成するか、ライブラリから探してきて使う

プログラムの例

宣言と同時に
初期化できる

```
int fib(int n)
{
    int a[45];
    int i = 2;
    a[0] = 0;
    a[1] = 1;
    while(i < 45){
        a[i] = a[i-1] + a[i-2];
        i = i + 1;
    }
    if(0 <= n && n < 45){
        return a[n];
    } else {
        return 0;
    }
}
```

さきほどの例より長い

文字と文字列

文字 `char` 型

- 文字を数値で表現（例えば `'H'` は `72` など）

文字列 `char` 型を要素に持つ配列

- 末尾に「`¥0`」（数値としては `0`）をつける
- よって長さ `n` の文字列を格納するのに長さ `n+1` の配列が必要

"Hello." =

'H'	'e'	'l'	'l'	'o'	'.'	¥0
-----	-----	-----	-----	-----	-----	----

注意 `'n'` と `"n"` は異なる意味

例：文字と数

```
#include <stdio.h>
int main()
{
    char a = 72;
    char b = 'H';
    printf("%d %d\n", a, b);
    printf("%c %c\n", a, b);
    if(a == b){
        printf("a=b\n");
    }
    return 0;
}
```

%d は数として表示
%c は文字として表示

例：文字列と配列

```
#include <stdio.h>
int main()
{
    char a[] = "Hello.";

    printf("%s\n", a);
    printf("%c %d\n", a[0], a[0]);
    printf("%c %d\n", a[1], a[1]);
    printf("%c %d\n", a[2], a[2]);
    return 0;
}
```

配列を文字列で初期化すると
コンパイラが長さを求めてくれる

%s は文字列の出力

その他の話題

- for 文
- ポインタ
- ポインタと配列

その他の話題

- **for 文**

- ポインタ

- ポインタと配列

for 文

繰り返しの記述に用いる

```
#include <stdio.h>
int main()
{
    char a[] = "Hello.¥n";
    int i;
    for(i = 1; i < 4; i++){
        printf("%c¥n", a[i]);
    }
    return 0;
}
```

i = 1, 2, 3 で繰り返す

for 文

繰り返しの記述に用いる

```
#include <stdio.h>
int main()
{
    char a[] = "Hello.¥n";
    int i;
    for(i = 1; i < 4; i++){
        printf("%c¥n", a[i]);
    }
    return 0;
}
```

```
#include <stdio.h>
int main()
{
    char a[] = "Hello.¥n";
    int i;
    i = 1;
    while(i < 4){
        printf("%c¥n", a[i]);
        i++;
    }
    return 0;
}
```

for 文

繰り返しの記述に用いる

```
#include <stdio.h>
int main()
{
    char a[] = "Hello.¥n";
    int i;
    for(i = 0; i < 6; i+=2){
        printf("%c¥n", a[i]);
    }
    return 0;
}
```

i = 0, 2, 4 で繰り返す

その他の話題

- for 文
- ポインタ
- ポインタと配列

ポインタ

記憶領域の場所を表すデータ

- 「`int *p;`」で `int` 型の記憶領域へのポインタ `p` を宣言
- 「`&a`」は変数 `a` の場所を指すポインタ
- 「`*p`」でポインタ `p` の指す場所にアクセス
 - 読み出し `x = *p;`
 - 書き込み `*p = 10;`

例

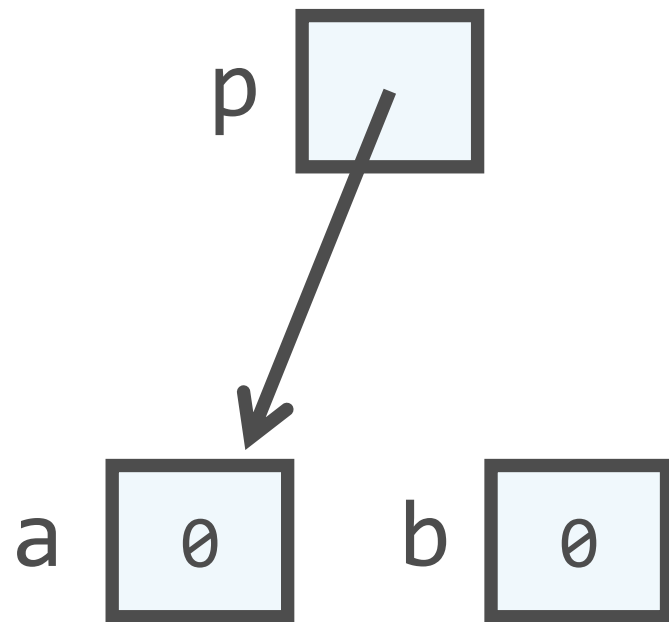
```
#include <stdio.h>
int main()
{
    int a=0, b=0;
    int *p;

    p = &a;
    *p = 10;
    printf("%d,%d,%d\n", *p, a, b);
    a = 20;
    printf("%d,%d,%d\n", *p, a, b);
    p = &b;
    printf("%d,%d,%d\n", *p, a, b);
    return 0;
}
```

例

```
#include <stdio.h>
int main()
{
    int a=0, b=0;
    int *p;

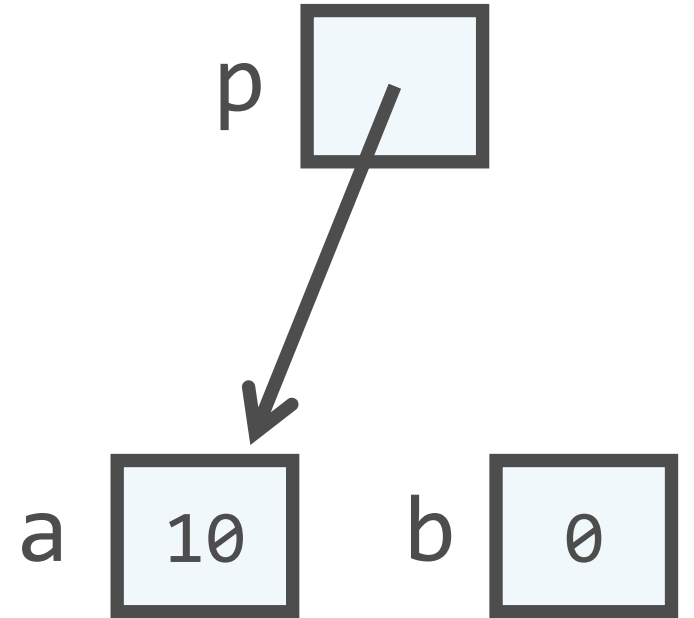
    p = &a;
    *p = 10;
    printf("%d,%d,%d\n", *p, a, b);
    a = 20;
    printf("%d,%d,%d\n", *p, a, b);
    p = &b;
    printf("%d,%d,%d\n", *p, a, b);
    return 0;
}
```



例

```
#include <stdio.h>
int main()
{
    int a=0, b=0;
    int *p;

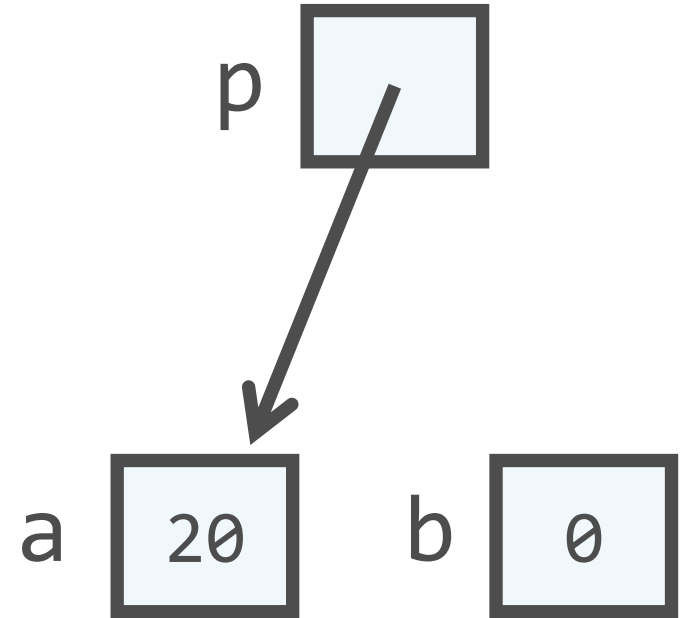
    p = &a;
    *p = 10;
    printf("%d,%d,%d\n", *p, a, b);
    a = 20;
    printf("%d,%d,%d\n", *p, a, b);
    p = &b;
    printf("%d,%d,%d\n", *p, a, b);
    return 0;
}
```



例

```
#include <stdio.h>
int main()
{
    int a=0, b=0;
    int *p;

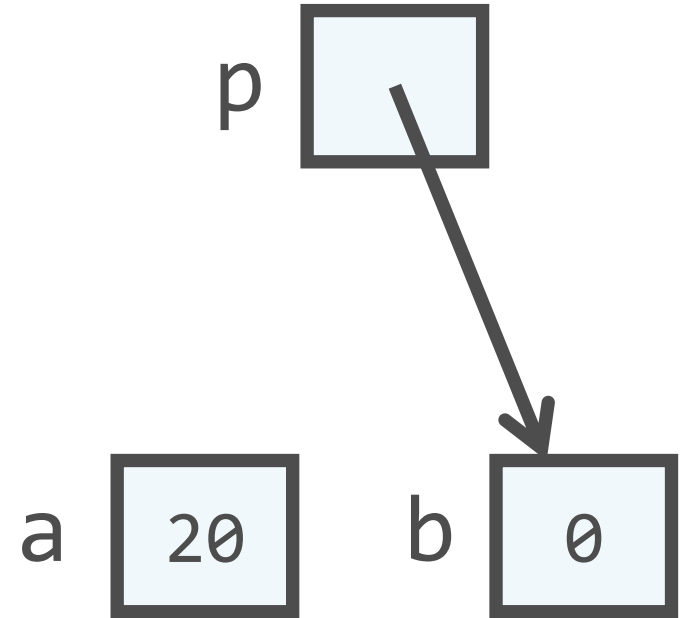
    p = &a;
    *p = 10;
    printf("%d,%d,%d\n", *p, a, b);
    a = 20;
    printf("%d,%d,%d\n", *p, a, b);
    p = &b;
    printf("%d,%d,%d\n", *p, a, b);
    return 0;
}
```



例

```
#include <stdio.h>
int main()
{
    int a=0, b=0;
    int *p;

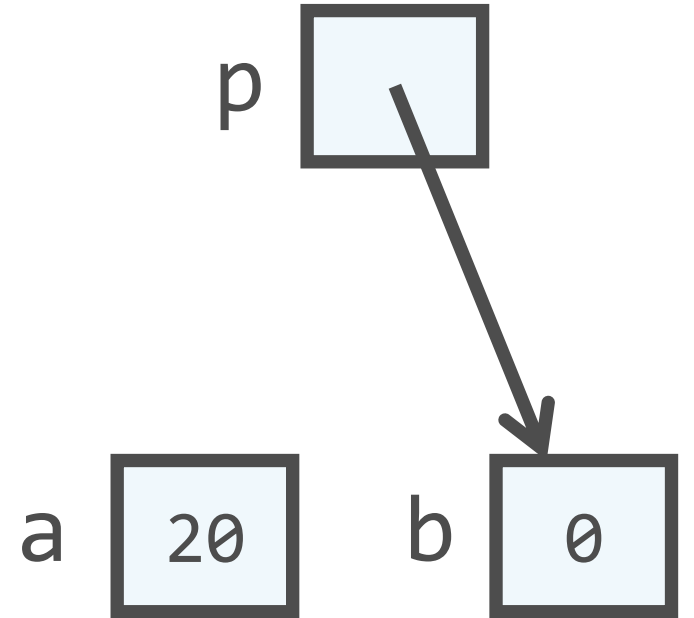
    p = &a;
    *p = 10;
    printf("%d,%d,%d\n", *p, a, b);
    a = 20;
    printf("%d,%d,%d\n", *p, a, b);
    p = &b;
    printf("%d,%d,%d\n", *p, a, b);
    return 0;
}
```



例

```
#include <stdio.h>
int main()
{
    int a=0, b=0;
    int *p;

    p = &a;
    *p = 10;
    printf("%d,%d,%d\n", *p, a, b);
    a = 20;
    printf("%d,%d,%d\n", *p, a, b);
    p = &b;
    printf("%d,%d,%d\n", *p, a, b);
    return 0;
}
```



クイズ

次のプログラムの出力はなにか。なぜそうなるのか

- 「(void*)」は型を無理やり合わせるためのおまじないです
- これがないと型が合わないという警告が出ます

```
#include <stdio.h>
int main()
{
    int a=1024;
    char *p;
    p = (void*)&a;
    *p = 0;
    printf("%d¥n", a);
    return 0;
}
```

注意

全員の挙動が同じとは
限りません

その他の話題

- for 文
- ポインタ
- **ポインタと配列**

ポインタと配列

文脈によっては 配列型の式は**先頭要素を指すポインタ**と解釈される

ポインタ p に整数を足すことができる

- ポインタ p が配列中の要素を指している場合
 $p+n$ は n 個先の要素を指すポインタ

するとポインタと配列は、ほぼ同列に扱うことができる

例

```
int fib(int n)
{
    int a[45];
    int i = 2;
    a[0] = 0;
    a[1] = 1;
    while(i < 45){
        a[i] = a[i-1] + a[i-2];
        i = i + 1;
    }
    if(0 <= n && n < 45){
        return a[n];
    } else {
        return 0;
    }
}
```



```
int fib(int n)
{
    int a[45];
    int *p = a;
    int i = 2;
    *(p+0) = 0;
    *(p+1) = 1;
    while(i < 45){
        *(p+i) = *(p+i-1) + *(p+i-2);
        i = i + 1;
    }
    if(0 <= n && n < 45){
        return *(p+n);
    } else {
        return 0;
    }
}
```

関数

基本的なところは Python と大きくは変わらない

- ただし内部定義は不可

C言語

```
int power(int x, int y)
{
    int i, r=1;
    for(i=0; i<y; i++){
        r *= x;
    }
    return r;
}
```

Python

```
def power(x,y):

    r = 1
    for i in range(y):
        r *= x

    return r
```

関数の宣言と定義

関数は使う前に宣言または定義をする

```
float sqroot(float);
```

使う前に
型だけ宣言

```
int main()
```

```
{
```

```
...
```

```
y = sqroot(x);
```

```
...
```

```
}
```

```
float sqroot(float x)
```

```
{
```

```
...
```

```
}
```

後で定義

```
float sqroot(float x)
```

```
{
```

```
...
```

```
}
```

```
int main()
```

```
{
```

```
...
```

```
y = sqroot(x);
```

```
...
```

```
}
```

使う前に定義

ポインタ引数

引数にポインタを取ると, 呼び出し元の変数の内容を変更できる

返り値に意味がない
ことを表す型

```
#include <stdio.h>
void swap(int *x, int *y)
{
    int w;
    w = *x; *x = *y; *y = w;
}
int main()
{
    int a = 1, b = 2;
    printf("a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```

ポインタ引数

引数にポインタを取ると, 呼び出し元の変数の内容を変更できる



```
#include <stdio.h>
void swap(int *x, int *y)
{
    int w;
    w = *x; *x = *y; *y = w;
}
int main()
{
    int a = 1, b = 2;
    printf("a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```



ポインタ引数

引数にポインタを取ると, 呼び出し元の変数の内容を変更できる

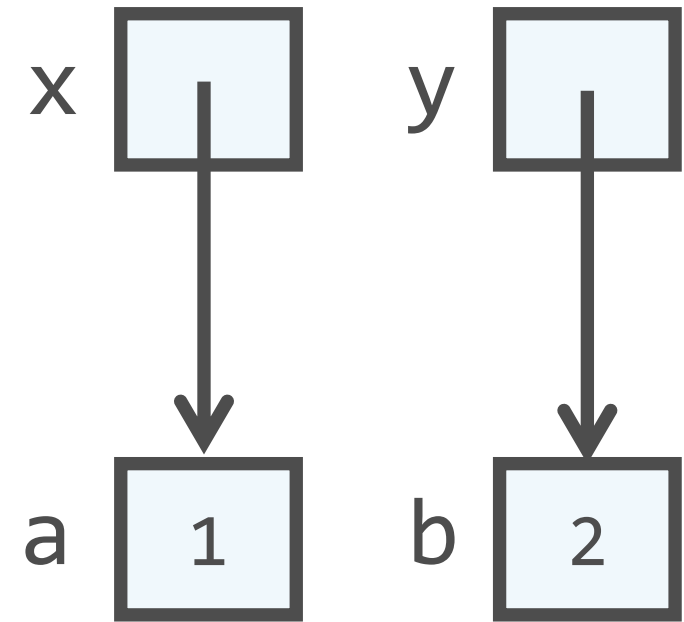


```
#include <stdio.h>
void swap(int *x, int *y)
{
    int w;
    w = *x; *x = *y; *y = w;
}
int main()
{
    int a = 1, b = 2;
    printf("a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```



ポインタ引数

引数にポインタを取ると, 呼び出し元の変数の内容を変更できる

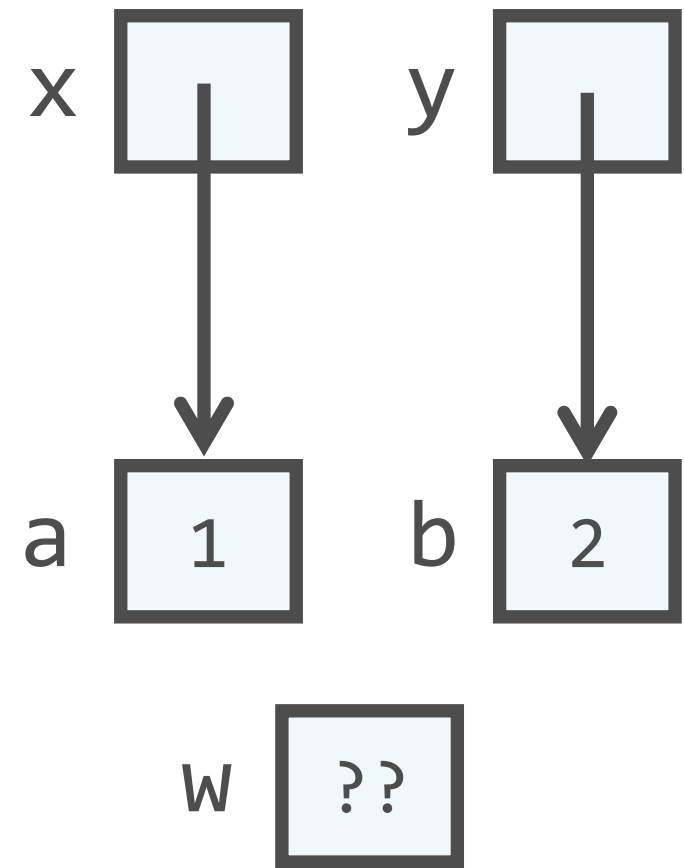


```
#include <stdio.h>
void swap(int *x, int *y)
{
    int w;
    w = *x; *x = *y; *y = w;
}
int main()
{
    int a = 1, b = 2;
    printf("a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```



ポインタ引数

引数にポインタを取ると, 呼び出し元の変数の内容を変更できる

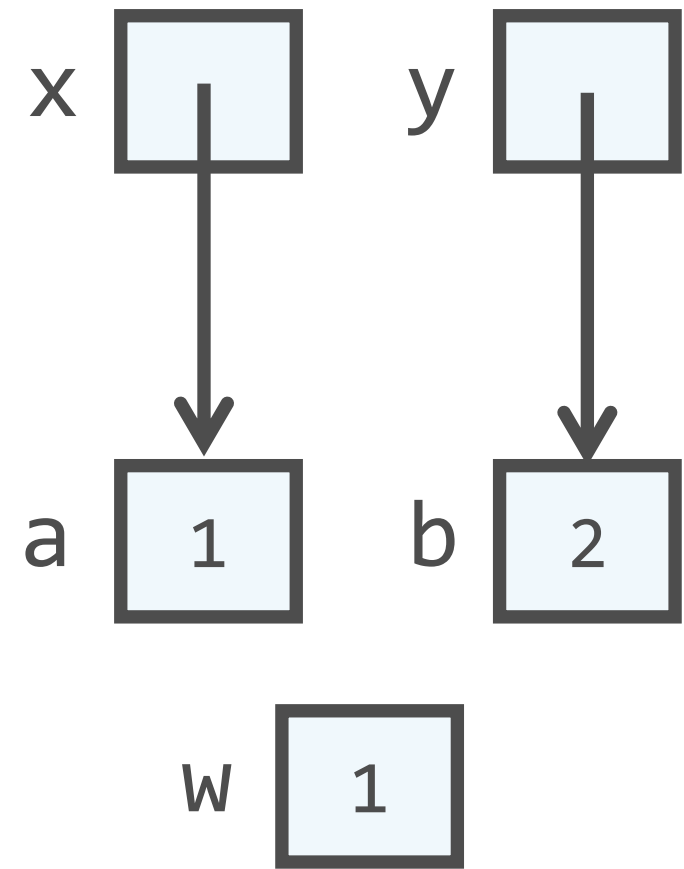


```
#include <stdio.h>
void swap(int *x, int *y)
{
    int w;
    w = *x; *x = *y; *y = w;
}
int main()
{
    int a = 1, b = 2;
    printf("a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```



ポインタ引数

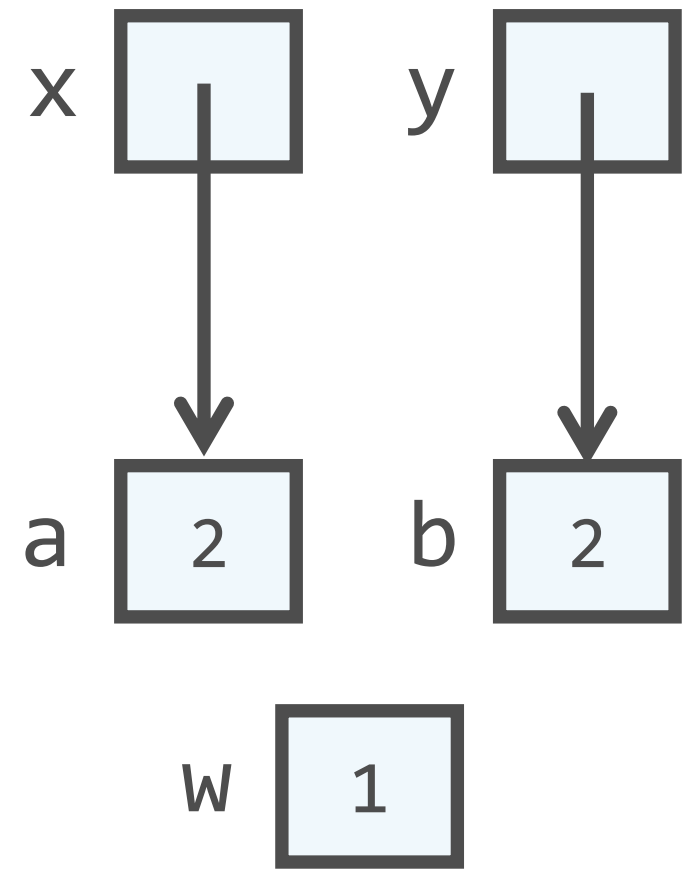
引数にポインタを取ると, 呼び出し元の変数の内容を変更できる



```
#include <stdio.h>
void swap(int *x, int *y)
{
    int w;
    w = *x; *x = *y; *y = w;
}
int main(
{
    int a = 1, b = 2;
    printf("a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```

ポインタ引数

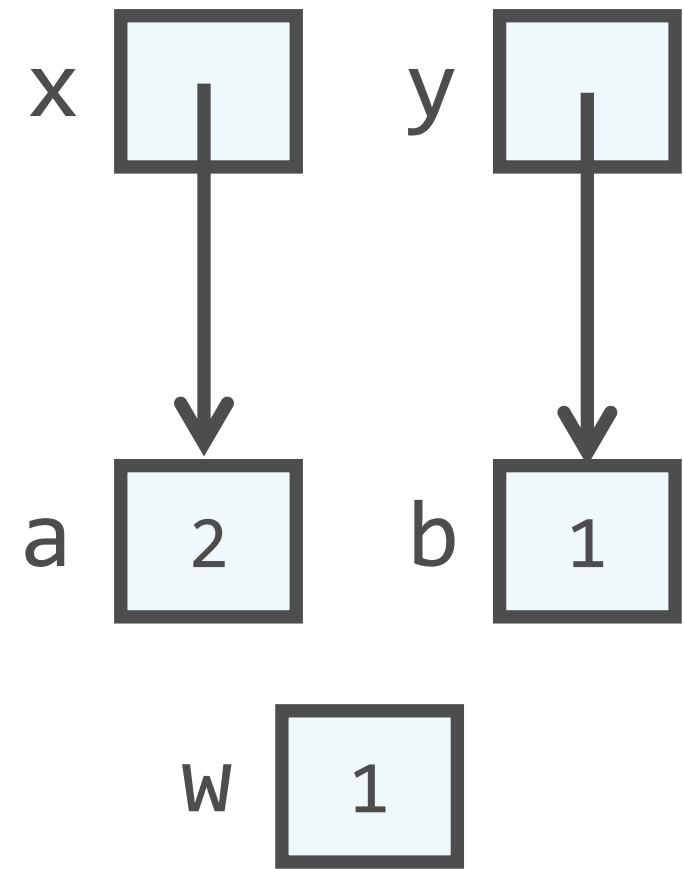
引数にポインタを取ると, 呼び出し元の変数の内容を変更できる



```
#include <stdio.h>
void swap(int *x, int *y)
{
    int w;
    w = *x; *x = *y; *y = w;
}
int main()
{
    int a = 1, b = 2;
    printf("a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```

ポインタ引数

引数にポインタを取ると, 呼び出し元の変数の内容を変更できる



```
#include <stdio.h>
void swap(int *x, int *y)
{
    int w;
    w = *x; *x = *y; *y = w;
}
int main()
{
    int a = 1, b = 2;
    printf("a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```

ポインタ引数

引数にポインタを取ると, 呼び出し元の変数の内容を変更できる



```
#include <stdio.h>
void swap(int *x, int *y)
{
    int w;
    w = *x; *x = *y; *y = w;
}
int main()
{
    int a = 1, b = 2;
    printf("a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```



配列とポインタ

配列は引数としては渡せない

- 「配列とポインタは似ている」ことを利用してポインタとして渡す

```
#include <stdio.h>
int sum(int a[], int n)
{
    int i, s=0;
    for(i=0; i<n; i++){
        s += a[i];
    }
    return s;
}
int main(){
    int b[10] = {1,2,3,4,5,6,7,8,9,10};
    printf("1+2+...+10=%d\n", sum(b,10));
    return 0;
}
```

配列のように書いているが、
a はポインタ

配列 b を
配列 b の先頭要素を指すポインタ
と同一視して関数に渡す

引数付きのマクロ

例 `#define abs(x) ((x)>=0 ? (x) : -(x))`

- 以降の `abs(...)` が `((...)>=0 ? (...) : -(...))` に置換される

単なる **文字列の置き換え** であることに注意

- 式や文の構造が意図せずに壊されるかもしれない

```
#define square(x) x * x
...
square(x+1) * y;
```

`x+1 * x+1 * y` に置換される
(つまり意味的には `x + x + y`)

- 十分に括弧を付けることで回避できることも

```
#define square(x) ((x) * (x))
```


引数付きのマクロ

例 `#define abs(x) ((x)>=0 ? (x) : -(x))`

- 以降の `abs(...)` が `((...)>=0 ? (...) : -(...))` に置換される

単なる **文字列の置き換え** であることに注意

- 式や文の構造が意図せずに壊されるかもしれない
 - 式が2回以上評価されてしまうかもしれない

```
#define square(x) ((x) * (x))
```

```
...
```

```
    square(x++) * y;
```

`((x++) * (x++)) * y` に置換される

構造体

Python の dataclass のようなもの (?)

C言語

```
struct person {  
    char* name;  
    int name_length;  
    int height;  
    int weight;  
    int age;  
};
```

Python

```
@dataclass  
class Person:  
    name: str  
    height: int  
    weight: int  
    age: int
```

構造体

Python の dataclass のようなもの (?)

C言語

```
struct complex {  
    float re;  
    float im;  
};
```

Python

```
@dataclass  
class Complex:  
    re: float  
    im: float
```

構造体

Python の dataclass のようなもの (?)

C言語

```
struct cell {  
    int info;  
    cell *next;  
};
```

Python

```
@dataclass  
class Cell:  
    info: int  
    next: Any
```

例：構造体

```
struct complex {  
    float re;  
    float im;  
};
```

構造体の定義
struct complex で型

引数にできる

```
struct complex product(struct complex x, struct complex y)  
{  
    struct complex r;  
    r.re = x.re * y.re - x.im * y.im;  
    r.im = x.re * y.im + x.im * y.re;  
    return r;  
}
```

.re や .im で中身にアクセスできる

返り値にできる

例：構造体

```
struct complex {  
    float re;  
    float im;  
};
```

構造体へのポインタ

```
struct complex product(struct complex *x, struct complex *y)  
{  
    struct complex r;  
    r.re = x->re * y->re - x->im * y->im;  
    r.im = x->re * y->im + x->im * y->re;  
    return r;  
}
```

->re や ->im で中身にアクセスできる

ヘッダファイル

関数宣言やマクロなどを集めたファイル

- `stdio.h` 標準入出力に関するもの
 - `printf` の宣言などが含まれる
- `math.h` 数学に関する関数や定数など
 - `sin` や `sqrt` といった関数の宣言

`#include` でヘッダファイルの取り込み

```
#include <stdio.h>
```

- 取り込みはプリプロセッサが行う

エラー？

右のファイルを用意して,

コンパイルしてみる

```
$ cc sqrt.c
```

(環境によっては) エラーが出る

sqrt.c

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main()
```

```
{
```

```
    printf("%f¥n", sqrt(2.0));
```

```
    return 0;
```

```
}
```

math.h で関数 **sqrt** は宣言されたが, 実体 (≒定義) がないため

- 実体を提供している **libm** とリンクする必要がある

```
$ cc sqrt.c -lm
```

- printf の実体は標準でリンクされるライブラリにあるため,
陽に指定しなくてもエラーにならなかった